COURS 2 — Entrées / Sorties, Opérateurs, et Structures Conditionnelles en C

Par Robert DIASSÉ

1. Rappel essentiel : Entrées et sorties en C

Le langage C utilise principalement deux fonctions pour communiquer avec l'utilisateur :

- `printf` : affichage
- `scanf` : lecture de valeurs

1.1 Affichage avec `printf`

`printf` écrit du texte ou des valeurs sur l'écran.

Format général :

```
printf("texte %code", valeurs);
```

Quelques codes fréquents :

Code	Signification
`%d`	entier (`int`)
`%f`	réel (`float`, `double`)
`%.2f`	réel affiché avec 2 décimales
`%c`	caractère
`%s`	chaîne de caractères

Exemple :

```
printf("Age = %d\n", 20);
printf("Note = %.2f\n", 13.75);
```

1.2 Lecture avec `scanf`

`scanf` lit une valeur tapée au clavier et la stocke dans une variable.

Format :

```
scanf("format", &variable);
```

L'opérateur `&` donne l'adresse de la variable, nécessaire pour que `scanf` puisse y stocker la valeur.

Exemples :

```
int age;
scanf("%d", &age);

double note;
scanf("%lf", &note);

char nom[20];
scanf("%19s", nom);     // lit un mot, max 19 caractères
```

2. Les opérateurs

Un opérateur applique une opération sur une ou plusieurs valeurs (appelées opérandes).

Une expression est une combinaison d'opérateurs et d'opérandes qui produit une valeur.
Les structures conditionnelles fonctionneront uniquement grâce à ces expressions.

2.1 Opérateurs arithmétiques

Opérateur	Signification
`+`	addition
`-`	soustraction
`*`	multiplication
`/`	division
`%`	reste de division entière (modulo)

Exemple :

```
int a = 7, b = 3;
int s  = a + b;   // 10
int d  = a - b;   // 4
int p  = a * b;   // 21
int q  = a / b;   // 2
int r  = a % b;   // 1
```

Diviser deux entiers donne un résultat entier.
Pour une division réelle :

```
double x = 7.0 / 3.0;
```

2.2 Affectation et formes combinées

`=` place une valeur dans une variable.

```
int x;
x = 5;
```

Formes abrégées :

```
x += 2;   // x = x + 2
x -= 3;   // x = x - 3
x *= 4;   // x = x * 4
x /= 2;   // x = x / 2
x %= 5;   // x = x % 5
```

2.3 Opérateurs de comparaison

Utilisés pour tester une relation entre deux valeurs.

Opérateur	Signification
`==`	égal à
`!=`	différent de
`<`	inférieur
`>`	supérieur
`<=`	inférieur ou égal
`>=`	supérieur ou égal

Exemples :

```
5 < 8   // vrai
5 == 8  // faux
```

Le résultat est 1 (vrai) ou 0 (faux).

> ⚠ Ne pas confondre :
> `=` affecte une valeur
> `==` compare deux valeurs

2.4 Opérateurs logiques

Permettent de combiner plusieurs conditions.

Opérateur	Signification
`&&`	ET logique
`||`	OU Logique
`!`	NON logique

Exemples :

```
(age >= 18 && age <= 35)
(note >= 10 || note == 9.5)
!(x == 0)
```

2.5 Priorité des opérateurs

Pour éviter les ambiguïtés, utiliser des parenthèses :

```
if ((a + b * c > 10) && (d == 0)) { ... }
```

---

3. Structures conditionnelles

Les structures conditionnelles permettent d'exécuter certaines instructions seulement si une condition est vraie.

3.1 Condition simple : `if`

```
if (condition) {
    instructions;
}
```

Exemple :

```
if (note >= 10) {
    printf("Admis\n");
}
```

3.2 Condition alternative : `if ... else`

```
if (condition) {
    instructions1;
} else {
    instructions2;
}
```

Exemple :

```
if (age >= 18) {
    printf("Majeur\n");
} else {
    printf("Mineur\n");
}
```

3.3 Conditions multiples : `if ... else if ... else`

```
if (note >= 16) {
    printf("Très bien\n");
}
else if (note >= 14) {
    printf("Bien\n");
}
else if (note >= 10) {
    printf("Passable\n");
}
else {
    printf("Échec\n");
}
```

3.4 Structure à choix multiple : `switch`

Utile quand une variable peut prendre plusieurs valeurs **connues**.

```
switch (expression) {
    case valeur1:
        instructions;
        break;
    case valeur2:
        instructions;
        break;
    ...
    default:
        instructions;
}
```

Exemple :

```
int jour;
scanf("%d", &jour);

switch (jour) {
    case 1: printf("Lundi\n"); break;
    case 2: printf("Mardi\n"); break;
    case 3: printf("Mercredi\n"); break;
    case 4: printf("Jeudi\n"); break;
    case 5: printf("Vendredi\n"); break;
    case 6: printf("Samedi\n"); break;
    case 7: printf("Dimanche\n"); break;
    default: printf("Valeur invalide\n");
}
```

4. Exercices d'application

Exercice 1

Lire deux entiers et afficher leur :

- somme
- différence
- produit
- quotient entier
- reste

Exercice 2

Lire un entier et afficher :

- “Zéro” si l’entier vaut 0
- “Positif” si > 0
- “Négatif” si < 0

Exercice 3

Lire une note (0 à 20) et afficher :

- A si note ≥ 16
- B si [14 ; 16[
- C si [10 ; 14[
- D si < 10

Exercice 4

Lire un âge et afficher :

- “Enfant” si < 12
- “Adolescent” si [12 ; 17]
- “Adulte” si ≥ 18

Exercice 5 (switch)

Lire un entier entre 1 et 7 et afficher le jour correspondant.